I'll analyze the FSDKC repository via GitHub (read-only) for architecture & design hotspots, then write the full report locally. Starting with stack detection and repo structure.Exploring the repository root and key manifests to detect the stack.Retrying repo listing and searching for Laravel/React structure from the commit history.Code search isn't indexed; exploring directories via file contents instead.Diving into controllers, modules, routes, and key source files for hotspot evidence.Fetching key source files and the existing audit doc for evidence.Computing precise KPIs, then writing the full report.Writing the full architecture report and pipeline artifacts.Writing the complete report to disk.Report saved to `docs/discovery/01-architecture-design.md` (full evidence + diagrams). Pipeline copy under `agent-runs/20260724T172631_idozya/`.

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Direct SQL/ORM in Controllers (H6), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Dual-Runtime Duplication (H10).

EXECUTIVE SUMMARY

Klearcom is a dual-runtime monolith (Laravel 12 API for Docker, Express `dev-api` for local `npm run dev`) with a React 19 SPA. Controllers and Express route handlers embed KPI math, tree building, and reachability formulas while calling Eloquent models / in-memory stores directly — there is no Repository layer and no Application Services for Discovery or Connect CRUD. Declared module folders (`backend/app/Modules/*`, `frontend/src/modules/*`) contain only `AGENTS.md` stubs, so Discovery and Connect are not real bounded contexts; Dashboard and Legacy reporting freely cross-read both domains and share Mongo collections via a `module` discriminator. The dominant risk is change amplification: the same business rules are duplicated across Laravel, Express, and page components, so a reachability or IVR-tree change must be patched in three places.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC avg (6 Laravel API controllers)	GOO
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 Eloquent access points in controllers (+ Express `store` in `server.js`)	HIG
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	44 ORM/Mongo access points outside any repository (0 repos)	HIG
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 cycles observed	GOO
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (`LegacyDataMapper` , `store.js` `buildTree`)	MOD

H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	~43% of query sites kept out of controllers (25/44 in controllers)	HIGH
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest: `server.js` 276 LOC)	GOO
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8+ (Dashboard, Legacy, RealTimeTestService, Mongo shared, frontend widgets)	HIGH
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	~38% (3/8 MariaDB+Mongo stores shared via `module` discriminator)	HIGH
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC avg (9 UI files); pages hold workflow	GOO
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 components hard-code paths on thin `api` client	GOO
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest: `ConnectPage.tsx` 222 LOC)	GOO
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels; small Zustand `uiStore` (2 IDs)	GOO
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (`LegacyMonitorPoller` class + `LegacyDashboardWidget` throw-without-boundary)	MOD
H10	Dual-Runtime Logic Duplication (additional)	Duplicated domain workflows across Laravel & Express	0	1–3	>3	≥5 (KPIs, tree, reachability, Discovery/Connect CRUD)	HIGH
H11	Anemic / Placeholder Bounded Contexts (additional)	Module dirs with no domain/application code	0	1–2	>2	4 module folders are `AGENTS.md` -only stubs	HIGH

1.4 Actions Required

Hotspot	Action	Rating	Priority

H2 Missing Service Layer	Extract `DashboardService`, `DiscoveryApplicationService`, `ConnectApplicationService`; move KPI, tree, and reachability logic out of controllers/ `server.js`	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces for jobs, monitors, checks, transcripts; bind Eloquent/Mongo implementations in the container	HIGH RISK	CRITICAL
H6 Direct SQL/ORM in Controllers	Ban Model imports from controllers; relocate all Eloquent/Mongo queries behind repositories	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Stop cross-importing Discovery/Connect models from Dashboard/Legacy; publish read APIs + ACL	HIGH RISK	HIGH
H9 Shared Database Coupling	Split or gate shared Mongo collections per context; remove cross-domain MariaDB reads	HIGH RISK	HIGH
H10 Dual-Runtime Duplication	Consolidate on one API runtime (or shared domain package) and add contract tests for Discovery/Connect	HIGH RISK	CRITICAL
H11 Anemic Module Boundaries	Move real code into `Modules/{Discovery,Connect}` namespaces on backend and frontend	HIGH RISK	HIGH
H5 Shared Utility Abuse	Replace `LegacyDataMapper` / `extract` and relocate `buildTree` into Discovery domain service	MODERATE	MEDIUM
F5 Legacy Component Patterns	Convert `LegacyMonitorPoller` to a hook with cleanup; add Error Boundaries; retire unsafe Legacy widget throws	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers and Express handlers become thin HTTP adapters; Discovery/Connect workflows are testable Application/Domain Services with a single reachability and IVR-tree implementation.
- Repository interfaces isolate MariaDB and Mongo persistence, enabling in-memory fakes and safer schema evolution without HTTP-layer churn.
- Real bounded contexts under `Modules/*` plus anti-corruption for Legacy/Dashboard stop hidden cross-domain coupling and shared-collection breakage.
- Collapsing dual-runtime duplication removes silent Laravel↔Express drift for local vs Docker environments.
- Frontend Error Boundaries and module API hooks reduce uncaught Legacy failures and prepare the SPA for growth without hard-coded path sprawl.